



SQL PROJECT – HOTEL BOOKING & GUESTS ANALYTICS

SUBMITTED BY – LOVISH KOTHARI

MODULE – SQL MINI PROJECT

PROJECT – SQL PROJECT HOTEL RATING ANALYTICS

PROJECT OVERVIEW

Analyze hotel bookings, guest behavior, room demand, additional services, payments, and customer reviews using SQL.

- 1. CREATE HOTEL BOOKING & GUESTS ANALYTICS DATABASE.**
- 2. INSERTED THE PROVIDED SAMPLE DATA.**
- 3. SOLVED 45 SQL BUSINESS QUERIES.**
- 4. INCLUDE OUTPUT SCREENSHOTS FOR EACH QUERY.**

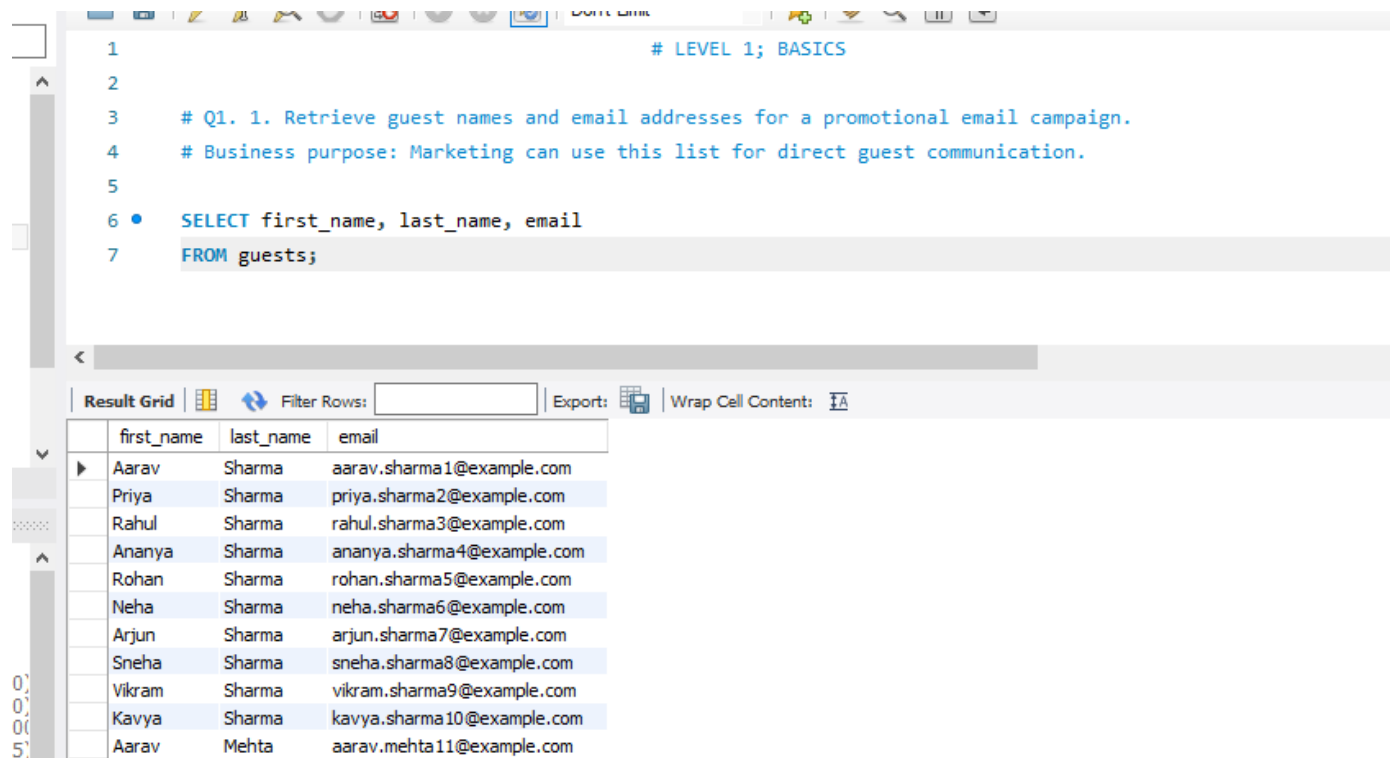
LEVEL 1: BASICS

Q1. 1. Retrieve guest names and email addresses for a promotional email campaign.

Business purpose: Marketing can use this list for direct guest communication.

```
SELECT first_name, last_name, email  
FROM guests;
```

OUTPUT(SCREENSHOT)



The screenshot shows a SQL IDE interface. The top pane displays the following SQL query:

```
1 # LEVEL 1; BASICS  
2  
3 # Q1. 1. Retrieve guest names and email addresses for a promotional email campaign.  
4 # Business purpose: Marketing can use this list for direct guest communication.  
5  
6 • SELECT first_name, last_name, email  
7 FROM guests;
```

The bottom pane shows the 'Result Grid' with the following data:

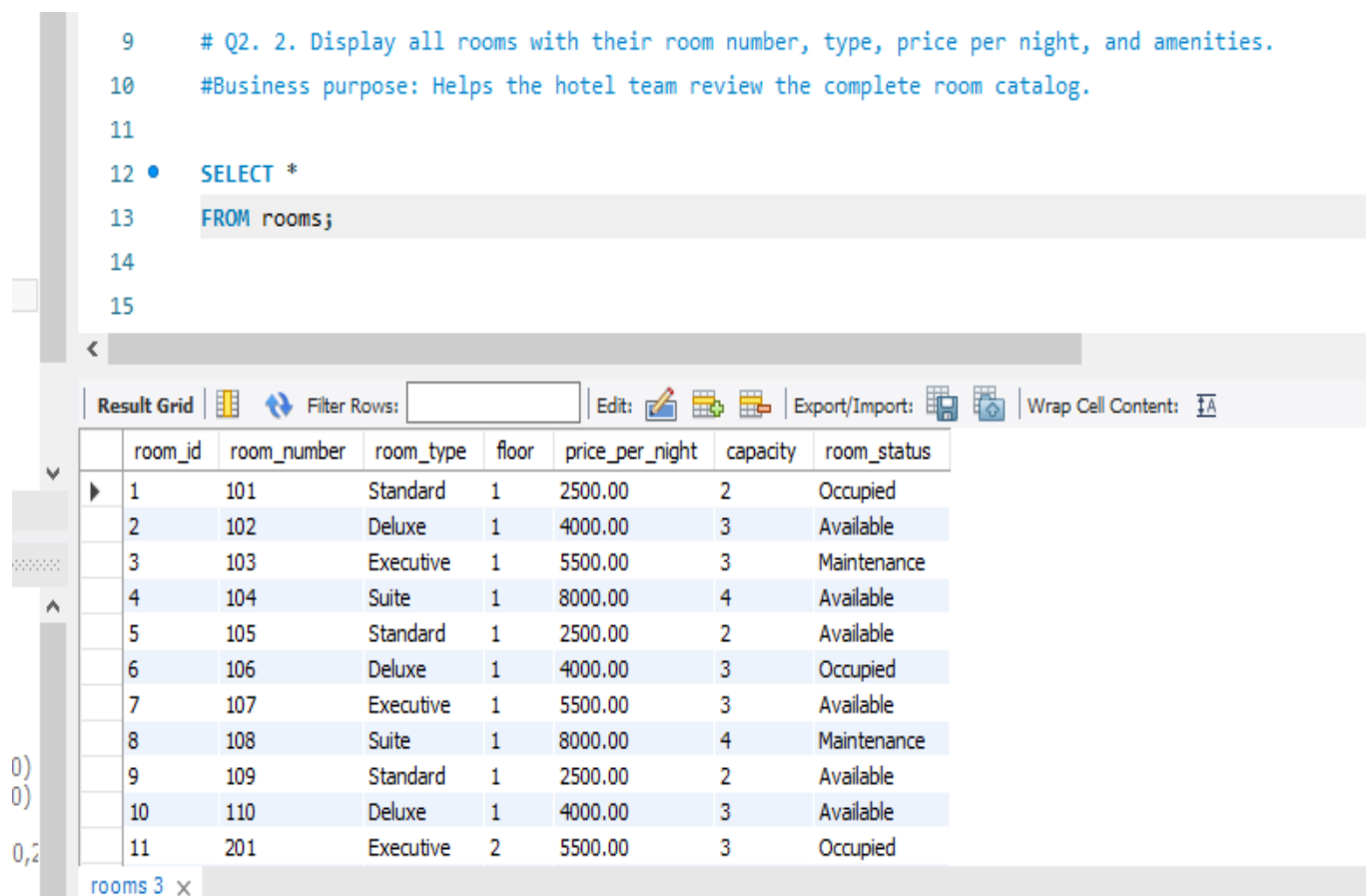
	first_name	last_name	email
▶	Aarav	Sharma	aarav.sharma1@example.com
	Priya	Sharma	priya.sharma2@example.com
	Rahul	Sharma	rahul.sharma3@example.com
	Ananya	Sharma	ananya.sharma4@example.com
	Rohan	Sharma	rohan.sharma5@example.com
	Neha	Sharma	neha.sharma6@example.com
	Arjun	Sharma	arjun.sharma7@example.com
	Sneha	Sharma	sneha.sharma8@example.com
	Vikram	Sharma	vikram.sharma9@example.com
	Kavya	Sharma	kavya.sharma10@example.com
	Aarav	Mehta	aarav.mehta11@example.com

Q2. Display all rooms with their room number, type, price per night, and amenities.

#Business purpose: Helps the hotel team review the complete room catalog.

```
SELECT *  
FROM rooms;
```

OUTPUT(SCREENSHOT)



9 # Q2. 2. Display all rooms with their room number, type, price per night, and amenities.
10 #Business purpose: Helps the hotel team review the complete room catalog.
11
12 • SELECT *
13 FROM rooms;
14
15

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:

	room_id	room_number	room_type	floor	price_per_night	capacity	room_status
▶	1	101	Standard	1	2500.00	2	Occupied
	2	102	Deluxe	1	4000.00	3	Available
	3	103	Executive	1	5500.00	3	Maintenance
	4	104	Suite	1	8000.00	4	Available
	5	105	Standard	1	2500.00	2	Available
	6	106	Deluxe	1	4000.00	3	Occupied
	7	107	Executive	1	5500.00	3	Available
	8	108	Suite	1	8000.00	4	Maintenance
	9	109	Standard	1	2500.00	2	Available
	10	110	Deluxe	1	4000.00	3	Available
	11	201	Executive	2	5500.00	3	Occupied

rooms 3 x

Q3. List all unique cities from which guests have registered.

#Business purpose: Useful for understanding the geographic origin of guests.

```
SELECT DISTINCT(city)
FROM guests;
```

OUTPUT(SCREENSHOT)

```
15      # Q3. List all unique cities from which guests have registered.
16      #Business purpose: Useful for understanding the geographic origin of guests.
17
18 •    SELECT DISTINCT(city)
19      FROM guests;
20
21
```

Result Grid	Filter Rows:	Export: 	Wrap Cell Content: 
city			
Jaipur			
Mumbai			
Delhi			
Pune			
Ahmedabad			
Bengaluru			
Hyderabad			
Chandigarh			
Kolkata			
Lucknow			
Dubai			
London			
Singapore			

Q4. Show all rooms with a price per night above n5,000.

#Business purpose: Helps identify premium rooms for high-value packages.

```
SELECT *  
  
FROM rooms  
  
WHERE price_per_night > 5000;
```

OUTPUT(SCREENSHOT)

21 # Q4. Show all rooms with a price per night above n5,000.
22 #Business purpose: Helps identify premium rooms for high-value packages.
23
24 • SELECT *
25 FROM rooms
26 WHERE price_per_night > 5000;
27
28

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	room_id	room_number	room_type	floor	price_per_night	capacity	room_status
▶	3	103	Executive	1	5500.00	3	Maintenance
	4	104	Suite	1	8000.00	4	Available
	7	107	Executive	1	5500.00	3	Available
	8	108	Suite	1	8000.00	4	Maintenance
	11	201	Executive	2	5500.00	3	Occupied
	12	202	Suite	2	8000.00	4	Available
	15	205	Executive	2	5500.00	3	Available
	16	206	Suite	2	8000.00	4	Occupied
	19	209	Executive	2	5500.00	3	Available
	20	210	Suite	2	8000.00	4	Available
	23	303	Executive	3	5500.00	3	Maintenance
	24	304	Suite	3	8000.00	4	Available
	27	307	Executive	3	5500.00	3	Available

rooms 6 x

Q5. Display rooms priced between n3,000 and n7,000 per night.

#Business purpose: Useful for creating a mid-range room offer.

```
SELECT *  
  
FROM rooms  
  
WHERE price_per_night BETWEEN 3000 AND 7000;
```

OUTPUT(SCREENSHOT)

```
28      # Q5. Display rooms priced between n3,000 and n7,000 per night.  
29      #Business purpose: Useful for creating a mid-range room offer.  
30  
31 •    SELECT *  
32      FROM rooms  
33      WHERE price_per_night BETWEEN 3000 AND 7000;  
34  
35
```

	room_id	room_number	room_type	floor	price_per_night	capacity	room_status
	2	102	Deluxe	1	4000.00	3	Available
	3	103	Executive	1	5500.00	3	Maintenance
	6	106	Deluxe	1	4000.00	3	Occupied
▶	7	107	Executive	1	5500.00	3	Available
	10	110	Deluxe	1	4000.00	3	Available
	11	201	Executive	2	5500.00	3	Occupied
	14	204	Deluxe	2	4000.00	3	Available
	15	205	Executive	2	5500.00	3	Available
	18	208	Deluxe	2	4000.00	3	Maintenance
	19	209	Executive	2	5500.00	3	Available
	22	302	Deluxe	3	4000.00	3	Available
	23	303	Executive	3	5500.00	3	Maintenance
	26	306	Deluxe	3	4000.00	3	Occupied

rooms 7 x

Q6. Retrieve bookings whose status is either 'Confirmed' or 'Checked-In'.

#Business purpose: Helps front-desk staff focus on active reservations.

```
SELECT *  
  
FROM bookings  
  
WHERE booking_status IN ('Confirmed', 'Checked-In');
```

OUTPUT(SCREENSHOT)

```
37  # Q6. Retrieve bookings whose status is either 'Confirmed' or 'Checked-In'.  
38  #Business purpose: Helps front-desk staff focus on active reservations.  
39  
40  •  SELECT *  
41  FROM bookings  
42  WHERE booking_status IN ('Confirmed', 'Checked-In');  
43
```

	booking_id	guest_id	room_id	check_in_date	check_out_date	number_of_guests	booking_status	total_amount	booking_source
	1	7	11	2025-01-03	2025-01-05	2	Checked-In	11330.00	Website
	4	28	4	2025-01-09	2025-01-14	1	Confirmed	44800.00	Website
	5	35	15	2025-01-11	2025-01-17	2	Checked-In	33000.00	Website
	8	56	8	2025-01-17	2025-01-20	1	Confirmed	26160.00	Website
▶	9	63	19	2025-01-19	2025-01-23	2	Checked-In	24640.00	Website
	12	84	12	2025-01-25	2025-01-26	1	Confirmed	8480.00	Website
	13	91	23	2025-01-27	2025-01-29	2	Checked-In	11990.00	Website
	16	112	16	2025-02-02	2025-02-07	1	Confirmed	41200.00	Website
	17	119	27	2025-02-04	2025-02-10	2	Checked-In	34980.00	Website
	20	140	20	2025-02-10	2025-02-13	1	Confirmed	24000.00	Website
	21	147	31	2025-02-12	2025-02-16	2	Checked-In	22660.00	Website
	24	18	24	2025-02-18	2025-02-19	1	Confirmed	8960.00	Website
	25	25	35	2025-02-20	2025-02-22	2	Checked-In	11000.00	Website

bookings13 x

Q7. Identify guests whose names start with the letter 'A'.

#Business purpose: Useful for practicing pattern matching and alphabetical segmentation.

```
SELECT *  
  
FROM guests  
  
WHERE first_name LIKE 'A%';
```

OUTPUT(SCREENSHOT)

44 # Q7. Identify guests whose names start with the letter 'A'.
45 #Business purpose: Useful for practicing pattern matching and alphabetical segmentation.
46
47 • SELECT *
48 FROM guests
49 WHERE first_name LIKE 'A%';
50
51

<

Result Grid

Filter Rows:

Edit

Export/Import

Wrap Cell Content

	guest_id	first_name	last_name	email	phone	city	country	registration_date
	107	Arjun	Agarwal	arjun.agarwal107@example.com	9000000107	Mumbai	India	2024-11-21
	131	Aarav	Bansal	aarav.bansal131@example.com	9000000131	Dubai	UAE	2025-02-01
	134	Ananya	Bansal	ananya.bansal134@example.com	9000000134	New York	USA	2025-02-10
▶	137	Arjun	Bansal	arjun.bansal137@example.com	9000000137	Mumbai	India	2025-02-19
	121	Aarav	Chopra	aarav.chopra121@example.com	9000000121	Jaipur	India	2025-01-02
	124	Ananya	Chopra	ananya.chopra124@example.com	9000000124	Pune	India	2025-01-11
	127	Arjun	Chopra	arjun.chopra127@example.com	9000000127	Hyderabad	India	2025-01-20
	41	Aarav	Gupta	aarav.gupta41@example.com	9000000041	Dubai	UAE	2024-05-07
	44	Ananya	Gupta	ananya.gupta44@example.com	9000000044	New York	USA	2024-05-16
	47	Arjun	Gupta	arjun.gupta47@example.com	9000000047	Mumbai	India	2024-05-25
	71	Aarav	Joshi	aarav.joshi71@example.com	9000000071	Dubai	UAE	2024-08-05
	74	Ananya	Joshi	ananya.joshi74@example.com	9000000074	New York	USA	2024-08-14
	77	Ariun	Joshi	ariun.ioshi77@example.com	9000000077	Mumbai	India	2024-08-23

guests 14 x

Q8. List Suite rooms costing less than n10,000 per night.

#Business purpose: Helps identify affordable premium-room options.

```
SELECT *  
  
FROM rooms  
  
WHERE room_type = 'Suite'  
and price_per_night < 10000;
```

OUTPUT(SCREENSHOT)

```
51 # Q8. List Suite rooms costing less than n10,000 per night.
52 #Business purpose: Helps identify affordable premium-room options.
53
54 ● SELECT *
55 FROM rooms
56 WHERE room_type = 'Suite'
57 and price_per_night < 10000;
58
```

[illegible]

Q9. Display room types and prices sorted from highest to lowest price.

#Business purpose: Makes premium rooms easy to compare.

```
SELECT room_type, price_per_night
FROM rooms
ORDER BY price_per_night DESC;
```

OUTPUT(SCREENSHOT)

```
50
59 # Q9. Display room types and prices sorted from highest to lowest price.
60 #Business purpose: Makes premium rooms easy to compare.
61
62 • SELECT room_type, price_per_night
63 FROM rooms
64 ORDER BY price_per_night DESC;
65
```

[illegible]

Q10. Display guest names and cities sorted by city and then by guest name.

#Business purpose: Useful for organized guest lists and reporting.

```
SELECT first_name, last_name, city
FROM guests
ORDER BY city ASC, first_name ASC;
```

OUTPUT(SCREENSHOT)

```
--
66  # Q10. Display guest names and cities sorted by city and then by guest name.
67  #Business purpose: Useful for organized guest lists and reporting.
68
69  • SELECT first_name, last_name, city
70      FROM guests
71      ORDER BY city ASC, first_name ASC;
72
```

<

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	first_name	last_name	city
	Kavya	Mehta	Ahmedabad
	Kavya	Joshi	Ahmedabad
	Rohan	Patel	Ahmedabad
	Rohan	Chopra	Ahmedabad
	Rohan	Sharma	Ahmedabad
	Rohan	Rao	Ahmedabad
	Rohan	Singh	Ahmedabad
	Aarav	Saxena	Bengaluru
▶	Aarav	Verma	Bengaluru
	Aarav	Kapoor	Bengaluru
	Aarav	Malhotra	Bengaluru
	Aarav	Khan	Bengaluru
	Neha	Chopra	Bengaluru

guests 18 x

LEVEL 2: FILTERING & FORMATTING

Q11. Find bookings where guest information is missing using a NULL check.

#Business purpose: Helps identify orphaned or incomplete booking records.

```
SELECT *
FROM bookings
WHERE guest_id IS NULL;
```

OUTPUT(SCREENSHOT)

[illegible]

Q12. Display guest names and cities using user-friendly column aliases.

#Business purpose: Useful when preparing readable reports.

```
SELECT

    first_name AS guest_first_name,

    last_name AS guest_last_name,

    city AS guest_city

FROM guests;
```

OUTPUT(SCREENSHOT)

```
85      # Q12. Display guest names and cities using user-friendly column aliases.
86      #Business purpose: Useful when preparing readable reports.
87
88 •    SELECT
89          first_name AS guest_first_name,
90          last_name AS guest_last_name,
91          city AS guest_city
92      FROM guests;
93
```

	guest_first_name	guest_last_name	guest_city
	Aarav	Sharma	Jaipur
	Priya	Sharma	Mumbai
▶	Rahul	Sharma	Delhi
	Ananya	Sharma	Pune
	Rohan	Sharma	Ahmedabad
	Neha	Sharma	Bengaluru
	Arjun	Sharma	Hyderabad
	Sneha	Sharma	Chandigarh
	Vikram	Sharma	Kolkata
	Kavya	Sharma	Lucknow
	Aarav	Mehta	Dubai

guests 21 ×

Q13. Calculate the estimated room charge for each booking as $\text{price_per_night} \times \text{nights}$.

#Business purpose: Provides a line-level room revenue calculation.

```
SELECT b.booking_id, r.price_per_night,  
       DATEDIFF(b.check_out_date, b.check_in_date) AS nights,  
       r.price_per_night * DATEDIFF(b.check_out_date,  
b.check_in_date) AS estimated_room_charge  
FROM bookings b  
JOIN rooms r  
ON b.room_id = r.room_id;
```

OUTPUT(SCREENSHOT)

```
94  # Q13. Calculate the estimated room charge for each booking as price_per_night * nights.  
95  #Business purpose: Provides a line-level room revenue calculation.  
96  
97  • SELECT b.booking_id, r.price_per_night,  
98         DATEDIFF(b.check_out_date, b.check_in_date) AS nights,  
99         r.price_per_night * DATEDIFF(b.check_out_date, b.check_in_date) AS estimated_room_charge  
100 FROM bookings b  
101 JOIN rooms r  
102 ON b.room_id = r.room_id;
```

< **Result Grid** Filter Rows: | Export: | Wrap Cell Content:

	booking_id	price_per_night	nights	estimated_room_charge
▶	11	2500.00	6	15000.00
	51	2500.00	4	10000.00
	91	2500.00	2	5000.00
	131	2500.00	6	15000.00
	171	2500.00	4	10000.00
	211	2500.00	2	5000.00
	251	2500.00	6	15000.00
	291	2500.00	4	10000.00
	331	2500.00	2	5000.00
	371	2500.00	6	15000.00

Result 22 ×

Q14. Combine guest name and city into one column called Guest_Location.

#Business purpose: Useful for compact guest summaries.

```
SELECT CONCAT(first_name, ' ',last_name, ',' ,city) AS
Guest_Location

FROM guests;
```

OUTPUT(SCREENSHOT)

```
104 # Q14. Combine guest name and city into one column called Guest_Location.
105 #Business purpose: Useful for compact guest summaries.
106
107 • SELECT CONCAT(first_name, ' ',last_name, ',' ,city) AS Guest_Location
108 FROM guests;
109
110
111
```

<

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

Guest_Location
Aarav Sharma,Jaipur
Priya Sharma,Mumbai
Rahul Sharma,Delhi
Ananya Sharma,Pune
Rohan Sharma,Ahmedabad
Neha Sharma,Bengaluru
Arjun Sharma,Hyderabad
Sneha Sharma,Chandigarh
Vikram Sharma,Kolkata
Kavya Sharma,Lucknow
Aarav Mehta,Dubai
Priya Mehta,London
Rahul Mehta,Singapore

Result 23 x

Q15. Extract only the month from each booking's check-in date.

#Business purpose: Helps prepare month-wise booking analysis.

```
SELECT booking_id, MONTH(check_in_date) AS Check_In_Month
FROM bookings;
```

OUTPUT(SCREENSHOT)

100

110 # Q15. Extract only the month from each booking's check-in date.

111 #Business purpose: Helps prepare month-wise booking analysis.

112

113 • SELECT booking_id, MONTH(check_in_date) AS Check_In_Month

114 FROM bookings;

115

<

Result Grid | Filter Rows: | Export: | Wrap Cell Content: IA

	booking_id	Check_In_Month
	8	1
	9	1
	10	1
	11	1
	12	1
	13	1
	14	1
	15	1
	16	2
	17	2
	18	2
	19	2
	20	2

Result 24

Q16. List all bookings that were cancelled.

#Business purpose: Useful for monitoring cancellations and potential lost revenue.

```
SELECT *  
  
FROM bookings  
  
WHERE booking_status = 'Cancelled';
```

OUTPUT(SCREENSHOT)

```
116 # Q16. List all bookings that were cancelled.  
117 #Business purpose: Useful for monitoring cancellations and potential lost revenue.  
118  
119 • SELECT *  
120 FROM bookings  
121 WHERE booking_status = 'Cancelled';  
122  
123  
124
```

	booking_id	guest_id	room_id	check_in_date	check_out_date	number_of_guests	booking_status	total_amount	booking_source
	2	14	22	2025-01-05	2025-01-08	3	Cancelled	12720.00	Website
	6	42	26	2025-01-13	2025-01-14	3	Cancelled	4120.00	Website
	10	70	30	2025-01-21	2025-01-26	3	Cancelled	20000.00	Website
▶	14	98	34	2025-01-29	2025-02-01	3	Cancelled	13440.00	Website
	18	126	38	2025-02-06	2025-02-07	3	Cancelled	4360.00	Website
	22	4	2	2025-02-14	2025-02-19	3	Cancelled	21200.00	Website
	26	32	6	2025-02-22	2025-02-25	3	Cancelled	12360.00	Website
	30	60	10	2025-03-02	2025-03-03	3	Cancelled	4000.00	Website
	34	88	14	2025-03-10	2025-03-15	3	Cancelled	22400.00	Website
	38	116	18	2025-03-18	2025-03-21	3	Cancelled	13080.00	Website
	42	144	22	2025-03-26	2025-03-27	3	Cancelled	4240.00	Website
	46	172	26	2025-04-02	2025-04-03	3	Cancelled	22600.00	Website

bookings25 x

Q17. Show bookings with a stay of 5 or more nights.

#Business purpose: Helps identify long-stay guests.

```
SELECT booking_id, check_in_date, check_out_date,
DATEDIFF(check_out_date, check_in_date) AS nights

FROM bookings

WHERE DATEDIFF(check_out_date, check_in_date) >= 5;
```

OUTPUT(SCREENSHOT)

123 # Q17. Show bookings with a stay of 5 or more nights.
124 #Business purpose: Helps identify long-stay guests.
125
126 • SELECT booking_id, check_in_date, check_out_date, DATEDIFF(check_out_date, check_in_date) AS nights
127 FROM bookings
128 WHERE DATEDIFF(check_out_date, check_in_date) >= 5;

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	booking_id	check_in_date	check_out_date	nights
✓ ▶	4	2025-01-09	2025-01-14	5
	5	2025-01-11	2025-01-17	6
	10	2025-01-21	2025-01-26	5
⋮	11	2025-01-23	2025-01-29	6
^	16	2025-02-02	2025-02-07	5
	17	2025-02-04	2025-02-10	6
	22	2025-02-14	2025-02-19	5
	23	2025-02-16	2025-02-22	6
	28	2025-02-26	2025-03-03	5
	29	2025-02-28	2025-03-06	6
	34	2025-03-10	2025-03-15	5
	35	2025-03-12	2025-03-18	6

Result 27 x

Q18. List guests who registered after 1 January 2025.

#Business purpose: Useful for analyzing recently acquired guests.

```
SELECT *  
  
FROM guests  
  
WHERE registration_date > '2025-01-01';
```

OUTPUT(SCREENSHOT)

130 # Q18. List guests who registered after 1 January 2025.
131 #Business purpose: Useful for analyzing recently acquired guests
132
133 • SELECT *
134 FROM guests |
135 WHERE registration_date > '2025-01-01';
136

<

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	guest_id	first_name	last_name	email	phone	city	country	registration_date
	121	Aarav	Chopra	aarav.chopra121@example.com	9000000121	Jaipur	India	2025-01-02
	122	Priya	Chopra	priya.chopra122@example.com	9000000122	Mumbai	India	2025-01-05
	123	Rahul	Chopra	rahul.chopra123@example.com	9000000123	Delhi	India	2025-01-08
	124	Ananya	Chopra	ananya.chopra124@example.com	9000000124	Pune	India	2025-01-11
	125	Rohan	Chopra	rohan.chopra125@example.com	9000000125	Ahmedabad	India	2025-01-14
	126	Neha	Chopra	neha.chopra126@example.com	9000000126	Bengaluru	India	2025-01-17
	127	Arjun	Chopra	arjun.chopra127@example.com	9000000127	Hyderabad	India	2025-01-20
	128	Sneha	Chopra	sneha.chopra128@example.com	9000000128	Chandigarh	India	2025-01-23
	129	Vikram	Chopra	vikram.chopra129@example.com	9000000129	Kolkata	India	2025-01-26
	130	Kavya	Chopra	kavya.chopra130@example.com	9000000130	Lucknow	India	2025-01-29
	131	Aarav	Bansal	aarav.bansal131@example.com	9000000131	Dubai	UAE	2025-02-01

guests 28 x

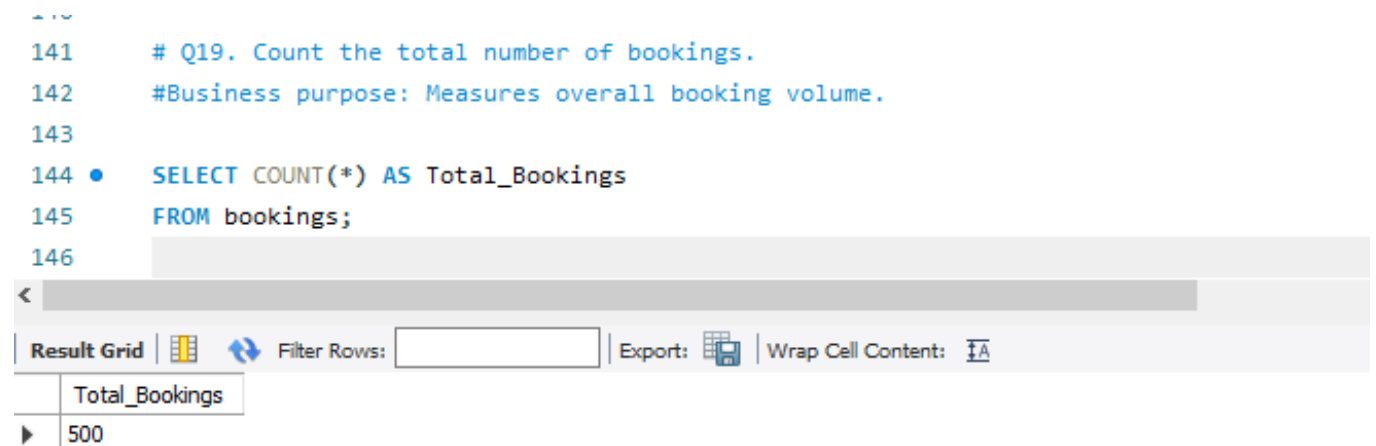
LEVEL 3: AGGREGATIONS

Q19. Count the total number of bookings.

#Business purpose: Measures overall booking volume.

```
SELECT COUNT(*) AS Total_Bookings
FROM bookings;
```

OUTPUT(SCREENSHOT)



```
141 # Q19. Count the total number of bookings.
142 #Business purpose: Measures overall booking volume.
143
144 • SELECT COUNT(*) AS Total_Bookings
145 FROM bookings;
146
```

Total_Bookings
500

Q20. Calculate the total booking revenue from all bookings.

#Business purpose: Provides the overall booked room value.

```
SELECT SUM(total_amount) AS Total_Booking_Revenue
FROM bookings;
```

OUTPUT(SCREENSHOT)

The screenshot shows a SQL query editor with the following content:

```
140
147 # Q20. Calculate the total booking revenue from all bookings.
148 #Business purpose: Provides the overall booked room value.
149
150 • SELECT SUM(total_amount) AS Total_Booking_Revenue
151 FROM bookings;
152
153
```

Below the query editor, there is a toolbar with the following options:

- Result Grid
- Filter Rows
- Export
- Wrap Cell Content

The output is displayed in a table with the following data:

Total_Booking_Revenue
9012070.00

Q21. Calculate the average booking value.

#Business purpose: Helps understand typical guest booking spend.

```
SELECT AVG(total_amount) AS Average_Booking_Value
FROM bookings;
```

OUTPUT(SCREENSHOT)

```
153
154   #Q21. Calculate the average booking value.
155   #Business purpose: Helps understand typical guest booking spend.
156
157 •   SELECT AVG(total_amount) AS Average_Booking_Value
158     FROM bookings;
159
```

<

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	Average_Booking_Value
▶	18024.140000

Result 33

Q22. Find the total number of unique guests who have made at least one booking.

#Business purpose: Measures the active guest base.

```
SELECT COUNT(DISTINCT guest_id) AS Unique_Booking_Guests
FROM bookings;
```

OUTPUT(SCREENSHOT)

The screenshot shows a SQL query editor with the following content:

```
100
161  #Q22. Find the total number of unique guests who have made at least one booking.
162  #Business purpose: Measures the active guest base.
163
164  •  SELECT COUNT(DISTINCT guest_id) AS Unique_Booking_Guests
165      FROM bookings;
166
```

Below the editor is a toolbar with the following options: Result Grid, Filter Rows, Export, and Wrap Cell Content. The Result Grid shows the following data:

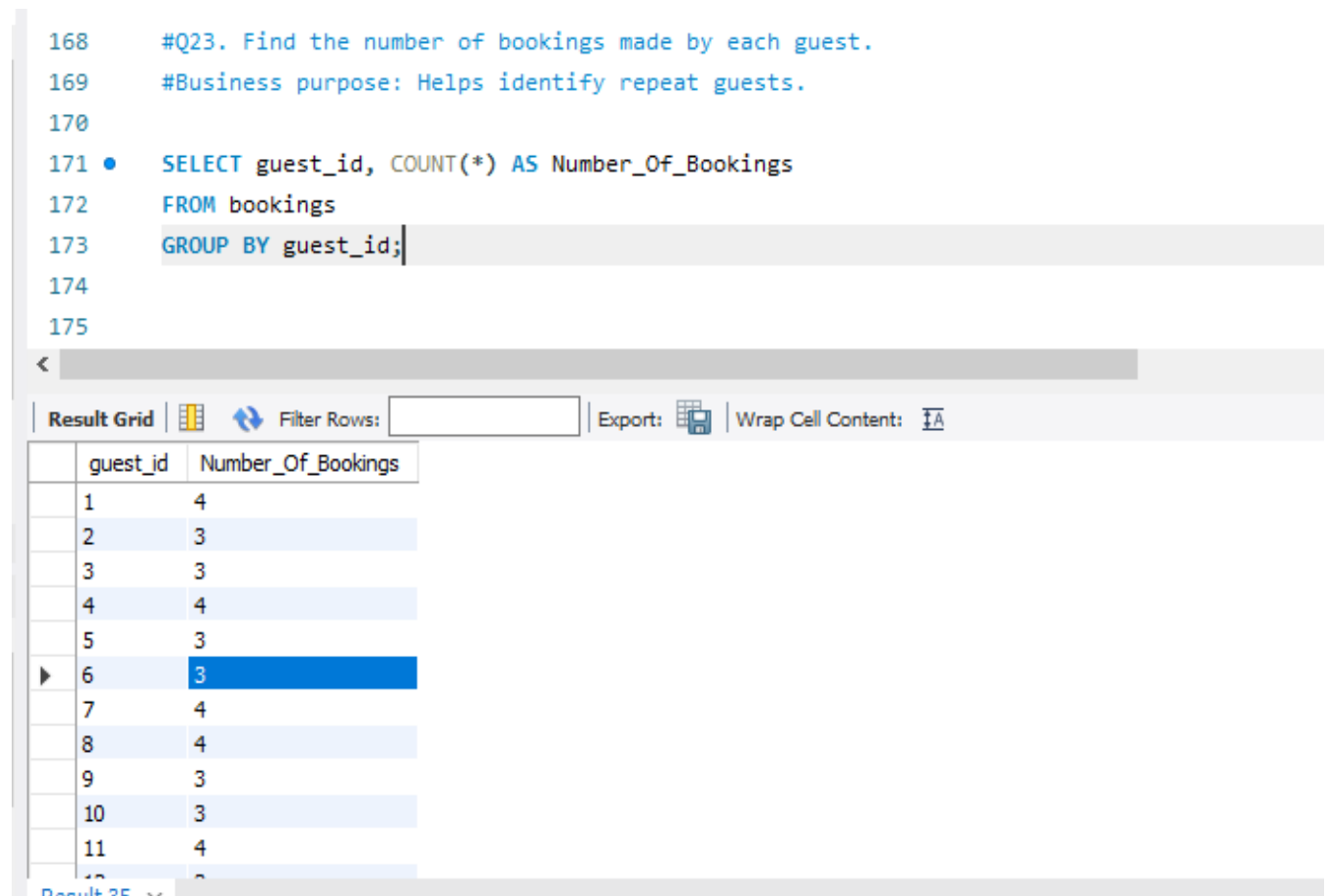
Unique_Booking_Guests
150

Q23. Find the number of bookings made by each guest.

#Business purpose: Helps identify repeat guests.

```
SELECT guest_id, COUNT(*) AS Number_Of_Bookings
FROM bookings
GROUP BY guest_id;
```

OUTPUT(SCREENSHOT)



168 #Q23. Find the number of bookings made by each guest.
169 #Business purpose: Helps identify repeat guests.
170
171 • SELECT guest_id, COUNT(*) AS Number_Of_Bookings
172 FROM bookings
173 GROUP BY guest_id;
174
175

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	guest_id	Number_Of_Bookings
	1	4
	2	3
	3	3
	4	4
	5	3
▶	6	3
	7	4
	8	4
	9	3
	10	3
	11	4
	12	4

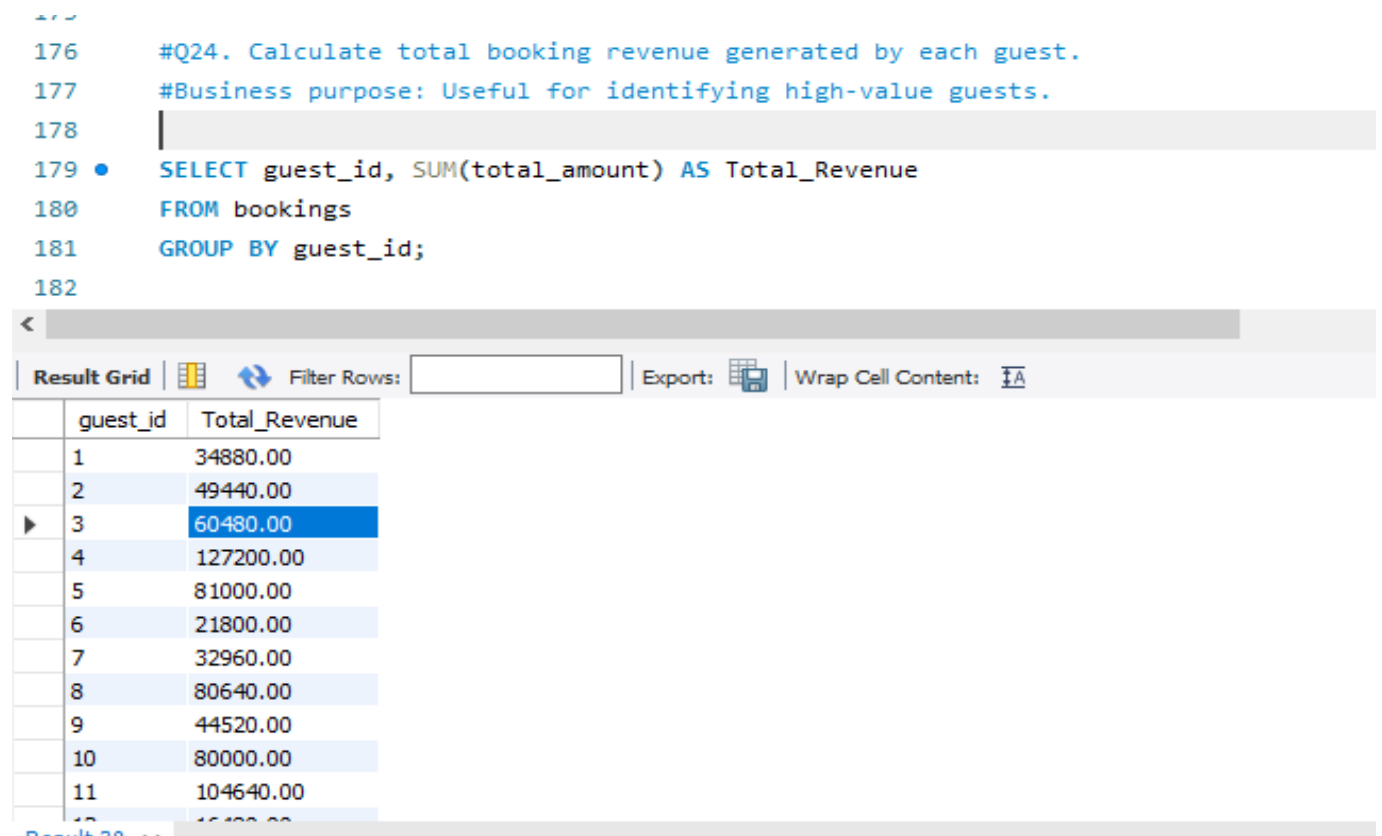
Default 35

Q24. Calculate total booking revenue generated by each guest.

#Business purpose: Useful for identifying high-value guests.

```
SELECT guest_id, SUM(total_amount) AS Total_Revenue
FROM bookings
GROUP BY guest_id;
```

OUTPUT(SCREENSHOT)



```
176 #Q24. Calculate total booking revenue generated by each guest.
177 #Business purpose: Useful for identifying high-value guests.
178
179 • SELECT guest_id, SUM(total_amount) AS Total_Revenue
180 FROM bookings
181 GROUP BY guest_id;
182
```

guest_id	Total_Revenue
1	34880.00
2	49440.00
3	60480.00
4	127200.00
5	81000.00
6	21800.00
7	32960.00
8	80640.00
9	44520.00
10	80000.00
11	104640.00
12	16400.00

Q25. Find the number of bookings for each room type.

#Business purpose: Helps compare demand across room categories.

```
SELECT r.room_type, COUNT(*) AS Number_Of_Bookings
FROM bookings b
JOIN rooms r
ON b.room_id = r.room_id
GROUP BY r.room_type;
```

OUTPUT9SCREENSHOT)

```
---
184     #Q25. Find the number of bookings for each room type.
185     #Business purpose: Helps compare demand across room categories.
186
187 •   SELECT r.room_type, COUNT(*) AS Number_Of_Bookings
188     FROM bookings b
189     JOIN rooms r
190     ON b.room_id = r.room_id
191     GROUP BY r.room_type;
192
193
```

<

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	room_type	Number_Of_Bookings
▶	Standard	125
	Deluxe	125
	Executive	125
	Suite	125

Q26. Calculate the average price per night for each room type.

#Business purpose: Useful for room-pricing comparisons.

```
SELECT room_type, AVG(price_per_night) AS Average_Price_Per_Night
FROM rooms
GROUP BY room_type;
```

OUTPUT(SCREENSHOT)

The screenshot shows a SQL query editor with the following code:

```
196
197     #Q26. Calculate the average price per night for each room type.
198     #Business purpose: Useful for room-pricing comparisons.
199
200 •   SELECT room_type, AVG(price_per_night) AS Average_Price_Per_Night
201     FROM rooms
202     GROUP BY room_type;
203
204
```

Below the editor is a toolbar with the following options: **Result Grid**, a grid icon, a refresh icon, **Filter Rows:** (with an empty input field), **Export:** (with a document icon), and **Wrap Cell Content:** (with a text icon).

The results grid displays the following data:

	room_type	Average_Price_Per_Night
▶	Standard	2500.000000
	Deluxe	4000.000000
	Executive	5500.000000
	Suite	8000.000000

Q27. Show the number of bookings made for each check-in date.

#Business purpose: Helps identify busy and quiet days.

```
SELECT check_in_date, COUNT(*) AS Number_Of_Bookings
FROM bookings
GROUP BY check_in_date
ORDER BY check_in_date;
```

OUTPUT(SCREENSHOT)

205
206 #Q27. Show the number of bookings made for each check-in date.
207 #Business purpose: Helps identify busy and quiet days.
208
209 • SELECT check_in_date, COUNT(*) AS Number_Of_Bookings
210 FROM bookings
211 GROUP BY check_in_date
212 ORDER BY check_in_date;
213
214

<

Result Grid  Filter Rows: Export:  Wrap Cell Content: 

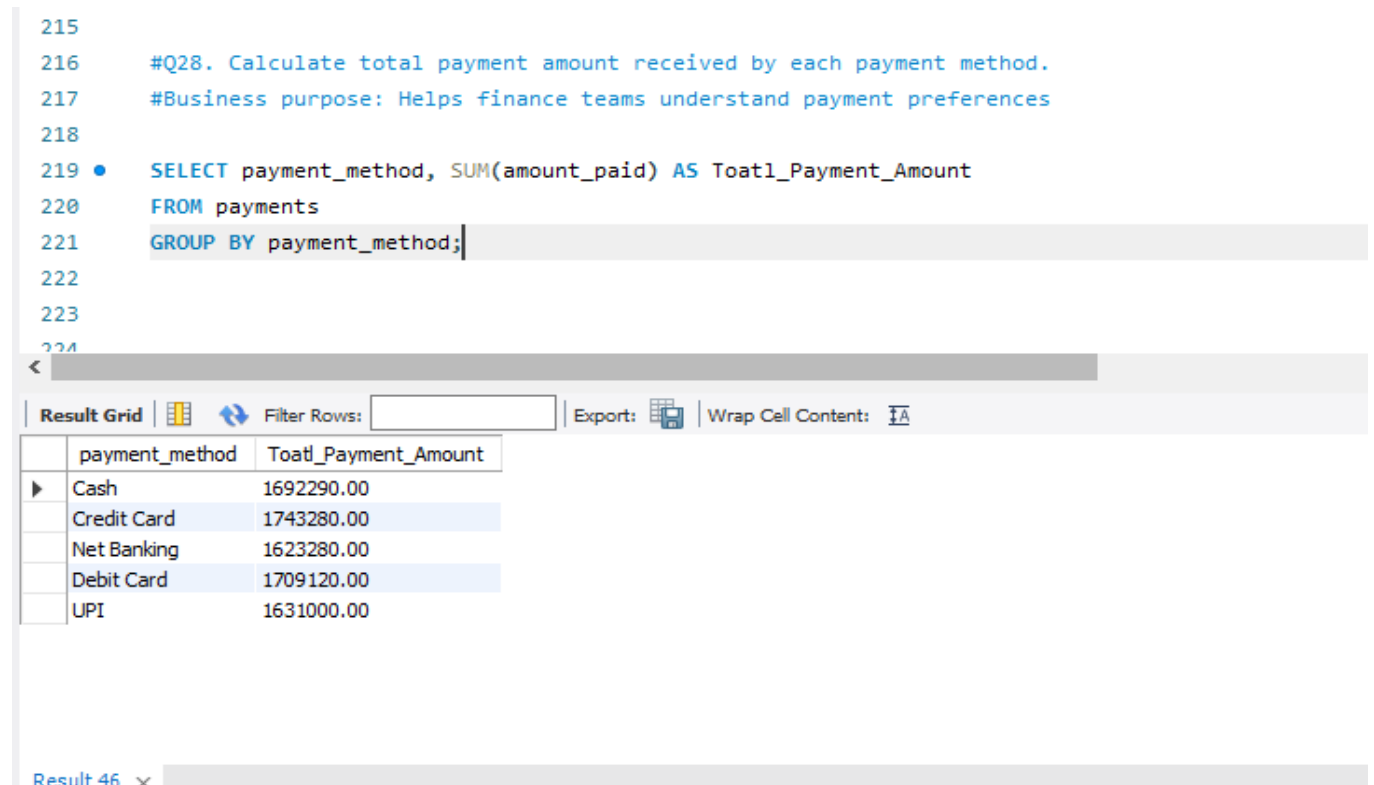
	check_in_date	Number_Of_Bookings
	2025-01-01	2
	2025-01-03	3
	2025-01-05	3
	2025-01-07	3
	2025-01-09	3
	2025-01-11	3
▶	2025-01-13	3
	2025-01-15	3
	2025-01-17	3
	2025-01-19	2

Q28. Calculate total payment amount received by each payment method.

#Business purpose: Helps finance teams understand payment preferences

```
SELECT payment_method, SUM(amount_paid) AS Total_Payment_Amount
FROM payments
GROUP BY payment_method;
```

OUTPUT(SCREENSHOT)



The screenshot displays a SQL query in a code editor and its corresponding results in a table. The query is as follows:

```
215
216 #Q28. Calculate total payment amount received by each payment method.
217 #Business purpose: Helps finance teams understand payment preferences
218
219 • SELECT payment_method, SUM(amount_paid) AS Total_Payment_Amount
220 FROM payments
221 GROUP BY payment_method;
```

Below the query, the results are shown in a table with the following columns: **payment_method** and **Total_Payment_Amount**.

payment_method	Total_Payment_Amount
Cash	1692290.00
Credit Card	1743280.00
Net Banking	1623280.00
Debit Card	1709120.00
UPI	1631000.00

The interface also includes a 'Result Grid' tab, a 'Filter Rows' input field, and buttons for 'Export' and 'Wrap Cell Content'.

LEVEL 4: MULTI-TABLE QUERIES (JOINS)

Q29. Retrieve booking details along with the guest name using an INNER JOIN.

#Business purpose: Shows who made each booking.

```
SELECT b.booking_id, CONCAT(g.first_name, ' ', g.last_name) AS Guest_name,  
       b.check_in_date, b.check_out_date, b.number_of_guests,  
       b.booking_status, b.total_amount, b.booking_source  
FROM bookings b  
INNER JOIN guests g  
ON b.guest_id = g.guest_id;
```

OUTPUT(SCREENSHOT)

219
220 # Q29. Retrieve booking details along with the guest name using an INNER JOIN.
221 #Business purpose: Shows who made each booking.
222
223 • SELECT b.booking_id, CONCAT(g.first_name, ' ', g.last_name) AS Guest_name,
224 b.check_in_date, b.check_out_date, b.number_of_guests, b.booking_status, b.total_amount, b.booking_source
225 FROM bookings b
226 INNER JOIN guests g
227 ON b.guest_id = g.guest_id;
228
229

<

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	booking_id	Guest_name	check_in_date	check_out_date	number_of_guests	booking_status	total_amount	booking_source
	43	Aarav Sharma	2025-03-28	2025-03-30	4	Completed	5450.00	Website
▶	193	Aarav Sharma	2026-01-22	2026-01-24	2	Checked-In	11990.00	Website
	343	Aarav Sharma	2025-09-24	2025-09-26	4	Completed	5450.00	Website
	493	Aarav Sharma	2025-05-27	2025-05-29	2	Checked-In	11990.00	Website
	86	Priya Sharma	2025-06-22	2025-06-25	3	Cancelled	12360.00	Website
	236	Priya Sharma	2025-02-22	2025-02-25	1	Confirmed	24720.00	Website
	386	Priya Sharma	2025-12-19	2025-12-22	3	Cancelled	12360.00	Website
	129	Rahul Sharma	2025-09-16	2025-09-20	2	Checked-In	24640.00	Website
	279	Rahul Sharma	2025-05-19	2025-05-23	4	Completed	11200.00	Website
	429	Rahul Sharma	2025-01-19	2025-01-23	2	Checked-In	24640.00	Website
	22	Ananya Sharma	2025-02-14	2025-02-19	3	Cancelled	21200.00	Website
	172	Ananya Sharma	2025-12-11	2025-12-16	1	Confirmed	42400.00	Website
	322	Ananya Sharma	2025-08-13	2025-08-18	3	Cancelled	21200.00	Website

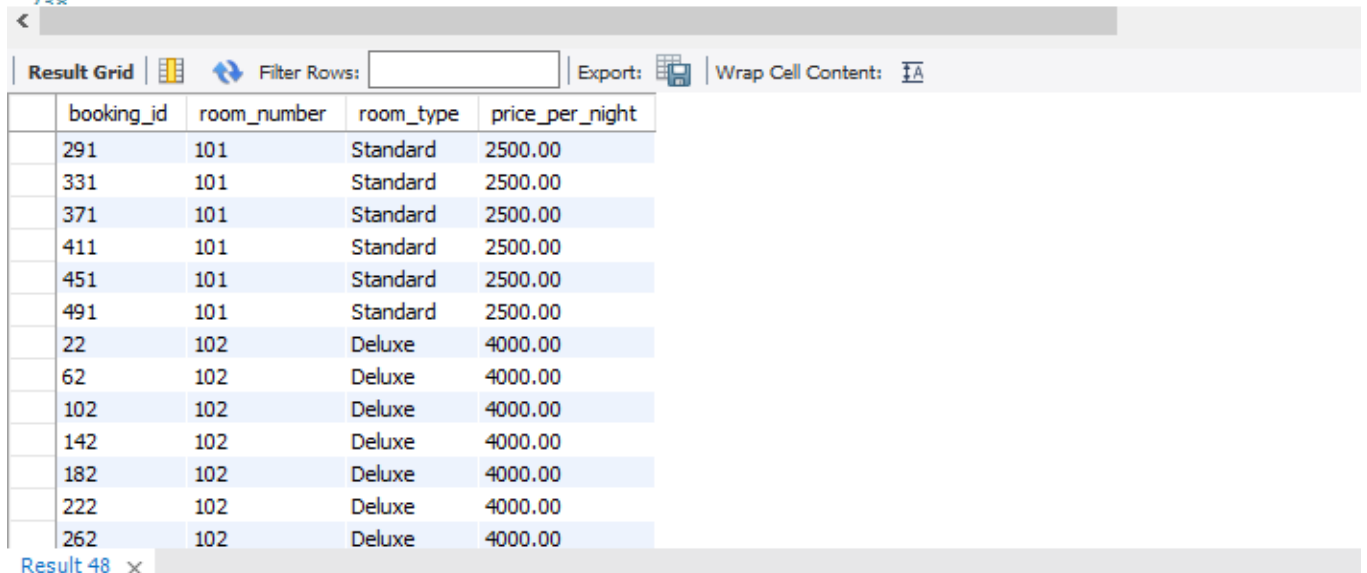
Q30. List all bookings with room number, room type, and nightly price.

#Business purpose: Helps front-desk and operations teams view reservation details.

```
SELECT b.booking_id, r.room_number, r.room_type, r.price_per_night
FROM bookings b
JOIN rooms r
ON b.room_id = r.room_id;
```

OUTPUT(SCREENSHOT)

```
229
230     #Q30. List all bookings with room number, room type, and nightly price.
231     #Business purpose: Helps front-desk and operations teams view reservation details.
232
233 •   SELECT b.booking_id, r.room_number, r.room_type, r.price_per_night
234     FROM bookings b
235     JOIN rooms r
236     ON b.room_id = r.room_id;
237
238
```



The screenshot shows a database query result grid with 48 rows. The columns are booking_id, room_number, room_type, and price_per_night. The data is grouped by room number and type. Rooms 101 and 102 are Standard and Deluxe respectively. The price per night is 2500.00 for Standard rooms and 4000.00 for Deluxe rooms.

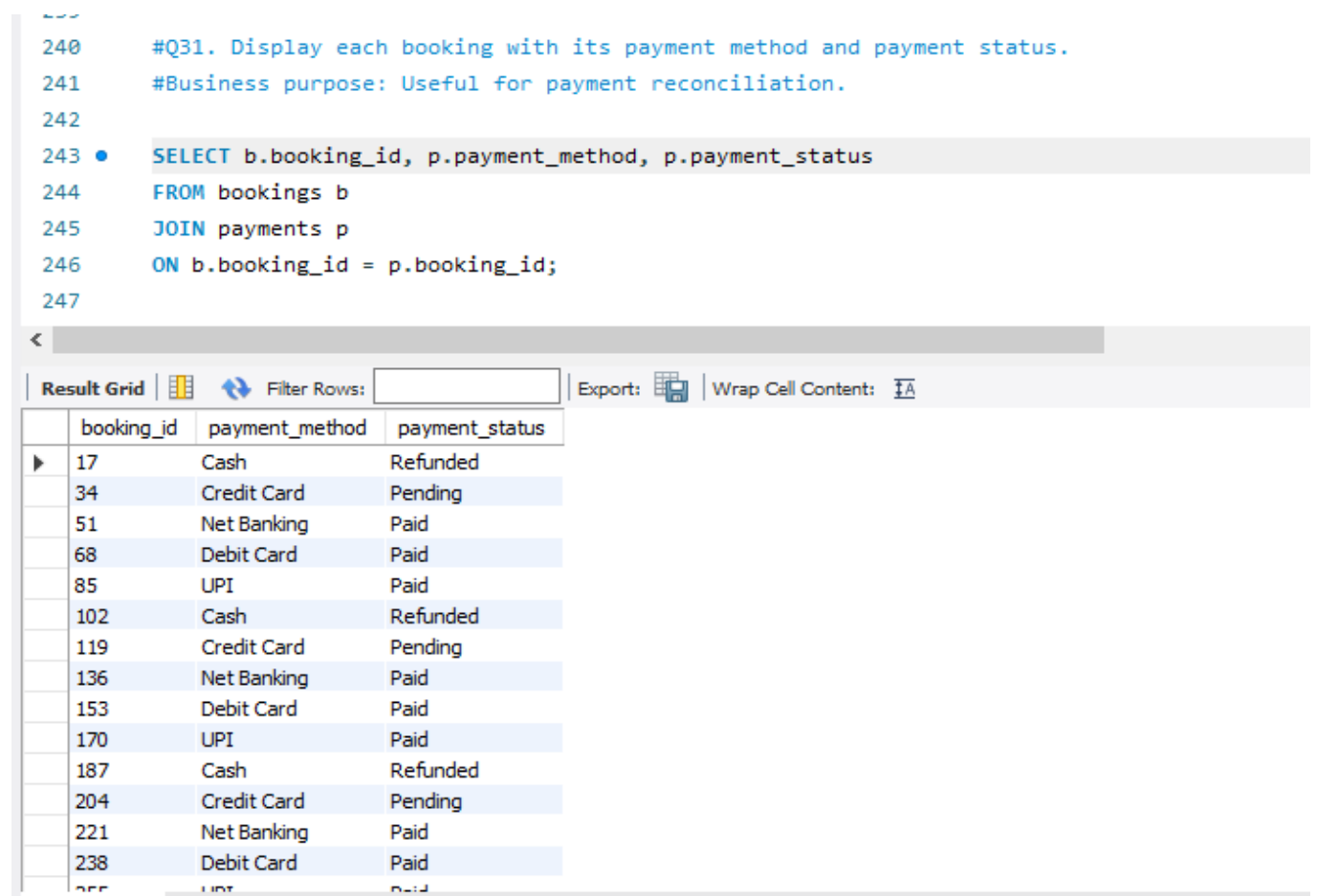
booking_id	room_number	room_type	price_per_night
291	101	Standard	2500.00
331	101	Standard	2500.00
371	101	Standard	2500.00
411	101	Standard	2500.00
451	101	Standard	2500.00
491	101	Standard	2500.00
22	102	Deluxe	4000.00
62	102	Deluxe	4000.00
102	102	Deluxe	4000.00
142	102	Deluxe	4000.00
182	102	Deluxe	4000.00
222	102	Deluxe	4000.00
262	102	Deluxe	4000.00

Result 48 x

Q31. Display each booking with its payment method and payment status. #Business purpose: Useful for payment reconciliation.

```
SELECT b.booking_id, p.payment_method, p.payment_status
FROM bookings b
JOIN payments p
ON b.booking_id = p.booking_id;
```

OUTPUT(SCREENSHOT)



The screenshot shows a SQL query editor with the following query:

```
240 #Q31. Display each booking with its payment method and payment status.
241 #Business purpose: Useful for payment reconciliation.
242
243 • SELECT b.booking_id, p.payment_method, p.payment_status
244 FROM bookings b
245 JOIN payments p
246 ON b.booking_id = p.booking_id;
247
```

Below the query editor, there is a toolbar with options: Result Grid, Filter Rows, Export, and Wrap Cell Content. The output is displayed in a table grid with the following data:

	booking_id	payment_method	payment_status
▶	17	Cash	Refunded
	34	Credit Card	Pending
	51	Net Banking	Paid
	68	Debit Card	Paid
	85	UPI	Paid
	102	Cash	Refunded
	119	Credit Card	Pending
	136	Net Banking	Paid
	153	Debit Card	Paid
	170	UPI	Paid
	187	Cash	Refunded
	204	Credit Card	Pending
	221	Net Banking	Paid
	238	Debit Card	Paid
	255	UPI	Paid

Q32. List all guests and their bookings using a LEFT JOIN.

#Business purpose: Shows both active bookers and guests with no bookings.

```
SELECT g.guest_id, CONCAT(g.first_name, ' ', g.last_name),  
       b.check_in_date, b.check_out_date, b.number_of_guests,  
       b.booking_status, b.total_amount, b.booking_source  
  
FROM guests g  
  
LEFT JOIN bookings b  
on g.guest_id = b.guest_id;
```

OUTPUT(SCREENSHOT)

252 #Q32. List all guests and their bookings using a LEFT JOIN.
253 #Business purpose: Shows both active bookers and guests with no bookings.
254
255 • SELECT g.guest_id, CONCAT(g.first_name, ' ', g.last_name),
256 b.check_in_date, b.check_out_date, b.number_of_guests, b.booking_status, b.total_amount, b.booking_sour
257 FROM guests g
258 LEFT JOIN bookings b
259 on g.guest_id = b.guest_id;
260

<

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	guest_id	CONCAT(g.first_name, ' ', g.last_name)	check_in_date	check_out_date	number_of_guests	booking_status	total_amount	booking_source
▶	1	Aarav Sharma	2025-03-28	2025-03-30	4	Completed	5450.00	Website
	1	Aarav Sharma	2026-01-22	2026-01-24	2	Checked-In	11990.00	Website
	1	Aarav Sharma	2025-09-24	2025-09-26	4	Completed	5450.00	Website
	1	Aarav Sharma	2025-05-27	2025-05-29	2	Checked-In	11990.00	Website
	2	Priya Sharma	2025-06-22	2025-06-25	3	Cancelled	12360.00	Website
	2	Priya Sharma	2025-02-22	2025-02-25	1	Confirmed	24720.00	Website
	2	Priya Sharma	2025-12-19	2025-12-22	3	Cancelled	12360.00	Website
	3	Rahul Sharma	2025-09-16	2025-09-20	2	Checked-In	24640.00	Website
	3	Rahul Sharma	2025-05-19	2025-05-23	4	Completed	11200.00	Website
	3	Rahul Sharma	2025-01-19	2025-01-23	2	Checked-In	24640.00	Website
	4	Ananya Sharma	2025-02-14	2025-02-19	3	Cancelled	21200.00	Website
	4	Ananya Sharma	2025-12-11	2025-12-16	1	Confirmed	42400.00	Website
	4	Ananya Sharma	2025-08-13	2025-08-18	3	Cancelled	21200.00	Website
	4	Ananya Sharma	2025-04-15	2025-04-20	1	Confirmed	42400.00	Website

Q33. List all rooms and any bookings associated with them using a LEFT JOIN.

#Business purpose: Helps identify rooms that have never been booked.

```
SELECT r.room_id, r.room_number, r.room_type, b.booking_id,
b.booking_status

FROM rooms r

LEFT JOIN bookings b

ON r.room_id = b.room_id;
```

OUTPUT(SCREENSHOT)

```
266 #Q33. List all rooms and any bookings associated with them using a LEFT JOIN.
267 #Business purpose: Helps identify rooms that have never been booked.
268
269 • SELECT r.room_id, r.room_number, r.room_type, b.booking_id, b.booking_status
270 FROM rooms r
271 LEFT JOIN bookings b
272 ON r.room_id = b.room_id;
273
```

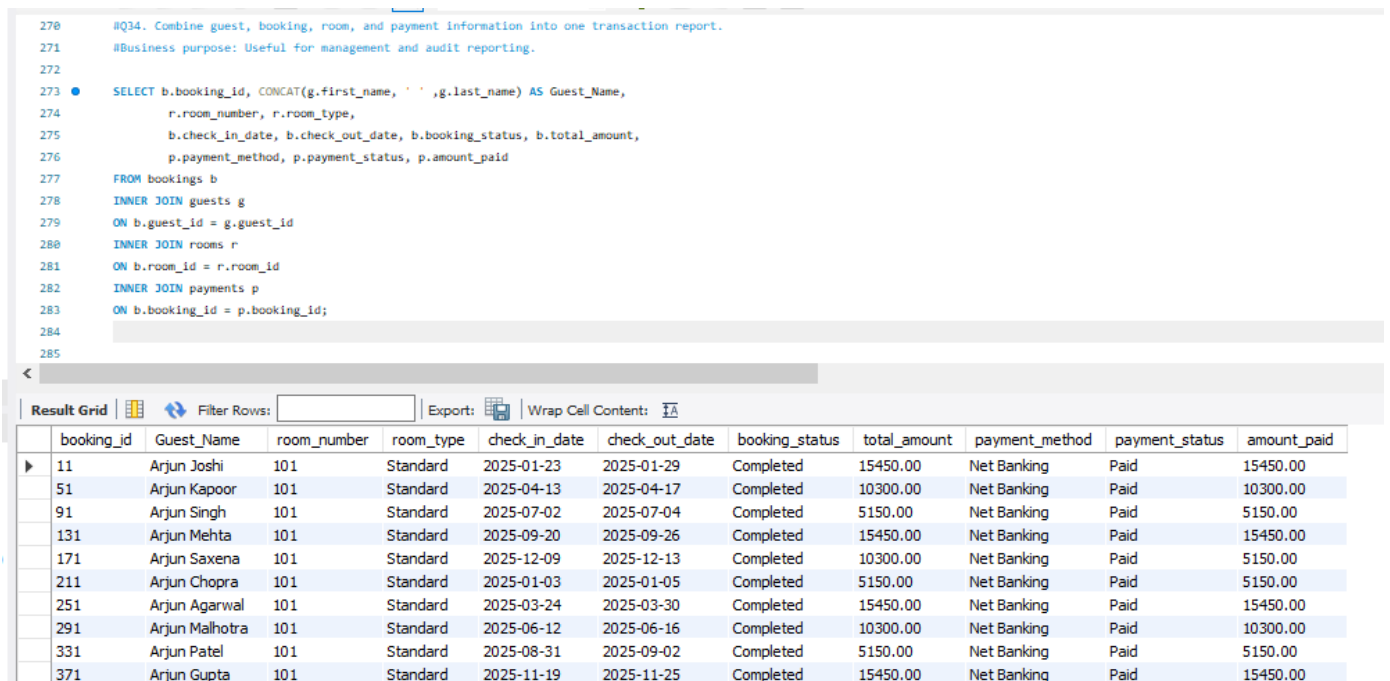
Result Grid					
Filter Rows:					
Export:					
Wrap Cell Content:					
	room_id	room_number	room_type	booking_id	booking_status
	1	101	Standard	11	Completed
	1	101	Standard	51	Completed
	1	101	Standard	91	Completed
	1	101	Standard	131	Completed
	1	101	Standard	171	Completed
	1	101	Standard	211	Completed
▶	1	101	Standard	251	Completed
	1	101	Standard	291	Completed
	1	101	Standard	331	Completed
	1	101	Standard	371	Completed
	1	101	Standard	411	Completed
	1	101	Standard	451	Completed
	1	101	Standard	491	Completed
	2	102	Deluxe	22	Cancelled
	2	102	Deluxe	62	Cancelled

Q34. Combine guest, booking, room, and payment information into one transaction report.

#Business purpose: Useful for management and audit reporting.

```
SELECT b.booking_id, CONCAT(g.first_name, ' ', g.last_name) AS Guest_Name,
       r.room_number, r.room_type, b.check_in_date, b.check_out_date,
       b.booking_status, b.total_amount, p.payment_method, p.payment_status, p.amount_paid
FROM bookings b
INNER JOIN guests g
ON b.guest_id = g.guest_id
INNER JOIN rooms r
ON b.room_id = r.room_id
INNER JOIN payments p
ON b.booking_id = p.booking_id;
```

OUTPUT(SCREENSHOT)



The screenshot shows a SQL query editor with the following code:

```
270 #Q34. Combine guest, booking, room, and payment information into one transaction report.
271 #Business purpose: Useful for management and audit reporting.
272
273 SELECT b.booking_id, CONCAT(g.first_name, ' ', g.last_name) AS Guest_Name,
274        r.room_number, r.room_type,
275        b.check_in_date, b.check_out_date, b.booking_status, b.total_amount,
276        p.payment_method, p.payment_status, p.amount_paid
277 FROM bookings b
278 INNER JOIN guests g
279 ON b.guest_id = g.guest_id
280 INNER JOIN rooms r
281 ON b.room_id = r.room_id
282 INNER JOIN payments p
283 ON b.booking_id = p.booking_id;
284
285
```

Below the query editor is a table with the following columns: booking_id, Guest_Name, room_number, room_type, check_in_date, check_out_date, booking_status, total_amount, payment_method, payment_status, amount_paid. The table contains 11 rows of data.

booking_id	Guest_Name	room_number	room_type	check_in_date	check_out_date	booking_status	total_amount	payment_method	payment_status	amount_paid
11	Arjun Joshi	101	Standard	2025-01-23	2025-01-29	Completed	15450.00	Net Banking	Paid	15450.00
51	Arjun Kapoor	101	Standard	2025-04-13	2025-04-17	Completed	10300.00	Net Banking	Paid	10300.00
91	Arjun Singh	101	Standard	2025-07-02	2025-07-04	Completed	5150.00	Net Banking	Paid	5150.00
131	Arjun Mehta	101	Standard	2025-09-20	2025-09-26	Completed	15450.00	Net Banking	Paid	15450.00
171	Arjun Saxena	101	Standard	2025-12-09	2025-12-13	Completed	10300.00	Net Banking	Paid	5150.00
211	Arjun Chopra	101	Standard	2025-01-03	2025-01-05	Completed	5150.00	Net Banking	Paid	5150.00
251	Arjun Agarwal	101	Standard	2025-03-24	2025-03-30	Completed	15450.00	Net Banking	Paid	15450.00
291	Arjun Malhotra	101	Standard	2025-06-12	2025-06-16	Completed	10300.00	Net Banking	Paid	10300.00
331	Arjun Patel	101	Standard	2025-08-31	2025-09-02	Completed	5150.00	Net Banking	Paid	5150.00
371	Arjun Gupta	101	Standard	2025-11-19	2025-11-25	Completed	15450.00	Net Banking	Paid	15450.00

Q35. Show booking services purchased by each guest, including service name and amount.

#Business purpose: Helps analyze guest spending beyond room charges.

```
SELECT CONCAT(g.first_name, ' ', g.last_name) AS Guest_name,
       b.booking_id, bs.service_id, bs.service_name, bs.quantity,
       bs.service_price, bs.service_date, bs.quantity * bs.service_price AS Service_Amount
FROM guests g
INNER JOIN bookings b
ON g.guest_id = b.guest_id
INNER JOIN booking_services bs
ON b.booking_id = bs.booking_id;
```

OUTPUT(SCREENSHOT)

286 #Q35. Show booking services purchased by each guest, including service name and amount.
287 #Business purpose: Helps analyze guest spending beyond room charges.
288
289 • SELECT CONCAT(g.first_name, ' ', g.last_name) AS Guest_name,
290 b.booking_id, bs.service_id, bs.service_name, bs.quantity,
291 bs.service_price, bs.service_date,
292 bs.quantity * bs.service_price AS Service_Amount
293 FROM guests g
294 INNER JOIN bookings b
295 ON g.guest_id = b.guest_id
296 INNER JOIN booking_services bs
297 ON b.booking_id = bs.booking_id;
298

<

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	Guest_name	booking_id	service_id	service_name	quantity	service_price	service_date	Service_Amount
▶	Aarav Sharma	43	311	Extra Bed	3	1500.00	2025-03-30	4500.00
	Aarav Sharma	193	361	Spa	2	1800.00	2026-01-23	3600.00
	Aarav Sharma	343	411	Airport Transfer	1	1200.00	2025-09-24	1200.00
	Aarav Sharma	493	461	Extra Bed	3	1500.00	2025-05-29	4500.00
	Priya Sharma	86	122	Laundry	3	700.00	2025-06-24	2100.00
	Priya Sharma	86	622	Room Service	2	900.00	2025-06-23	1800.00
	Priya Sharma	236	172	Room Service	2	900.00	2025-02-23	1800.00
	Priya Sharma	386	222	Breakfast	1	500.00	2025-12-19	500.00
	Rahul Sharma	129	433	Spa	2	1800.00	2025-09-17	3600.00
	Rahul Sharma	279	483	Airport Transfer	1	1200.00	2025-05-19	1200.00

Q36. Display reviews with guest name, room type, rating, and review text.

#Business purpose: Useful for customer-experience analysis.

```
SELECT re.review_id, CONCAT(g.first_name, ' ', g.last_name) AS Guest_name,
       r.room_type, re.rating, re.review_text
FROM reviews re
INNER JOIN guests g
ON re.guest_id = g.guest_id
INNER JOIN rooms r
ON re.room_id = r.room_id;
```

OUTPUT(SCREENSHOT)

```
305 #Q36. Display reviews with guest name, room type, rating, and review text.
306 #Business purpose: Useful for customer-experience analysis.
307
308 • SELECT re.review_id, CONCAT(g.first_name, ' ', g.last_name) AS Guest_name,
309       r.room_type, re.rating, re.review_text
310 FROM reviews re
311 INNER JOIN guests g
312 ON re.guest_id = g.guest_id
313 INNER JOIN rooms r
314 ON re.room_id = r.room_id;
```

	review_id	Guest_name	room_type	rating	review_text
	23	Arjun Kapoor	Standard	4	Room was good but service was slow.
	63	Arjun Khan	Standard	4	Room was good but service was slow.
	103	Arjun Verma	Standard	4	Room was good but service was slow.
▶	143	Arjun Malhotra	Standard	4	Room was good but service was slow.
	183	Arjun Saxena	Standard	4	Room was good but service was slow.
	223	Arjun Kapoor	Standard	4	Room was good but service was slow.
	6	Ananya Kapoor	Deluxe	3	The room was spacious and clean.
	46	Ananya Khan	Deluxe	3	The room was spacious and clean.
	86	Ananya Verma	Deluxe	3	The room was spacious and clean.
	126	Ananya Malhotra	Deluxe	3	The room was spacious and clean.
	166	Ananya Saxena	Deluxe	3	The room was spacious and clean.
	206	Ananya Kapoor	Deluxe	3	The room was spacious and clean.

Result 54 ×

Q37. Find guests who have both a booking and a review using multiple joins.

#Business purpose: Helps identify highly engaged guests.

```
SELECT DISTINCT g.guest_id, CONCAT(g.first_name, ' ', g.last_name) AS
Guest_name

FROM guests g

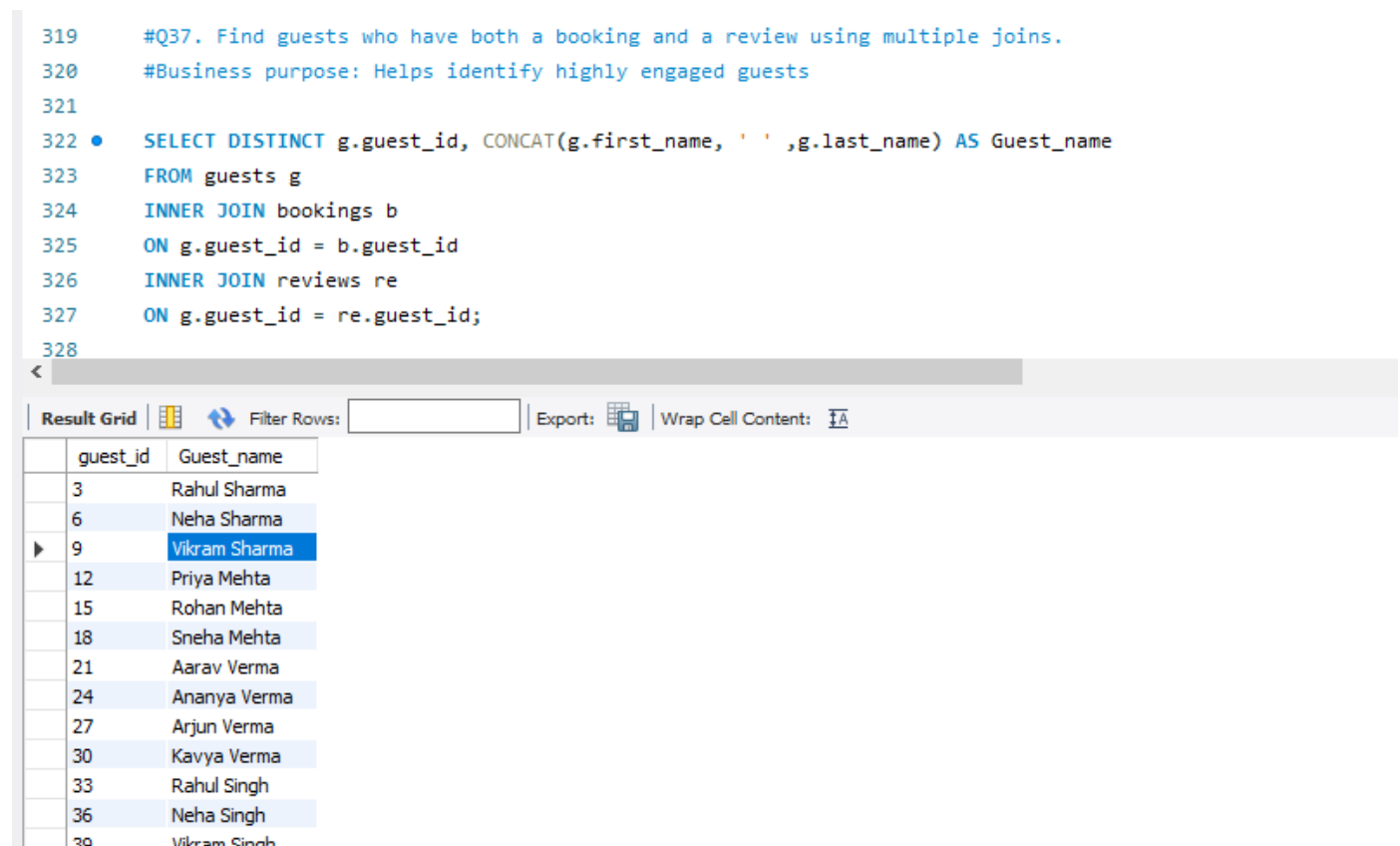
INNER JOIN bookings b

ON g.guest_id = b.guest_id

INNER JOIN reviews re

ON g.guest_id = re.guest_id;
```

OUTPUT(SCREENSHOT)



The screenshot displays a SQL query editor with a query window and a result grid. The query window shows the following SQL code:

```
319 #Q37. Find guests who have both a booking and a review using multiple joins.
320 #Business purpose: Helps identify highly engaged guests
321
322 • SELECT DISTINCT g.guest_id, CONCAT(g.first_name, ' ', g.last_name) AS Guest_name
323 FROM guests g
324 INNER JOIN bookings b
325 ON g.guest_id = b.guest_id
326 INNER JOIN reviews re
327 ON g.guest_id = re.guest_id;
328
```

The result grid shows the output of the query, displaying a list of guests with their IDs and names. The table has two columns: guest_id and Guest_name. The data is as follows:

guest_id	Guest_name
3	Rahul Sharma
6	Neha Sharma
9	Vikram Sharma
12	Priya Mehta
15	Rohan Mehta
18	Sneha Mehta
21	Aarav Verma
24	Ananya Verma
27	Arjun Verma
30	Kavya Verma
33	Rahul Singh
36	Neha Singh
39	Vikram Singh

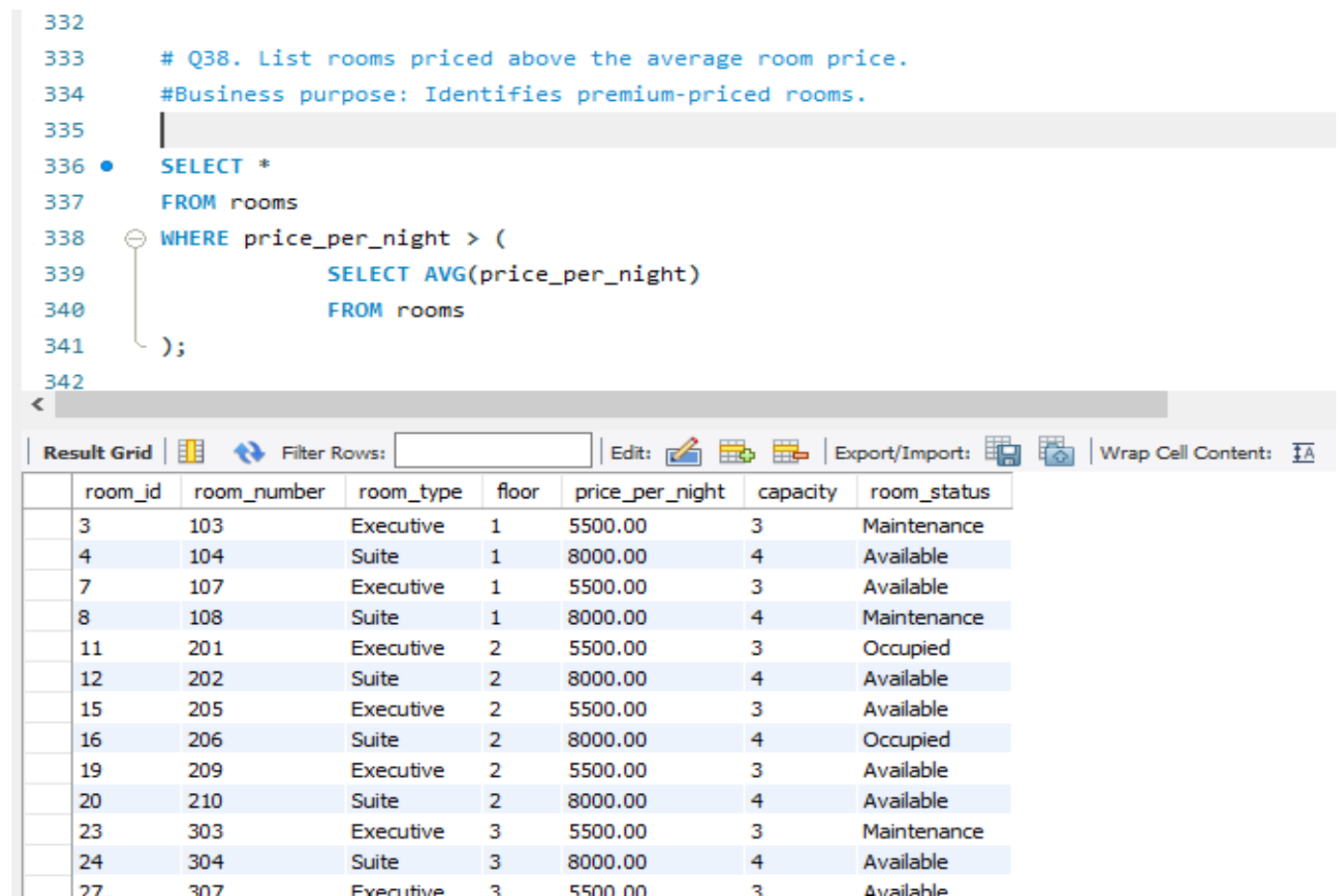
LEVEL 5: SUBQUERIES

Q38. List rooms priced above the average room price.

#Business purpose: Identifies premium-priced rooms.

```
SELECT *  
  
FROM rooms  
  
WHERE price_per_night > (  
        SELECT AVG(price_per_night)  
        FROM rooms);
```

OUTPUT(SCREENSHOT)



The screenshot displays a SQL query editor with the following text:

```
332  
333 # Q38. List rooms priced above the average room price.  
334 #Business purpose: Identifies premium-priced rooms.  
335  
336 • SELECT *  
337 FROM rooms  
338 WHERE price_per_night > (  
339     SELECT AVG(price_per_night)  
340     FROM rooms  
341 );  
342
```

Below the query editor is a toolbar with options: Result Grid, Filter Rows, Edit, Export/Import, and Wrap Cell Content. The Result Grid shows the following data:

	room_id	room_number	room_type	floor	price_per_night	capacity	room_status
	3	103	Executive	1	5500.00	3	Maintenance
	4	104	Suite	1	8000.00	4	Available
	7	107	Executive	1	5500.00	3	Available
	8	108	Suite	1	8000.00	4	Maintenance
	11	201	Executive	2	5500.00	3	Occupied
	12	202	Suite	2	8000.00	4	Available
	15	205	Executive	2	5500.00	3	Available
	16	206	Suite	2	8000.00	4	Occupied
	19	209	Executive	2	5500.00	3	Available
	20	210	Suite	2	8000.00	4	Available
	23	303	Executive	3	5500.00	3	Maintenance
	24	304	Suite	3	8000.00	4	Available
	27	307	Executive	3	5500.00	3	Available

Q39. Find guests who have made at least one booking using EXISTS.

#Business purpose: Identifies active guests.

```
SELECT g.guest_id, CONCAT(g.first_name, ' ', g.last_name) AS
Guest_name

FROM guests g

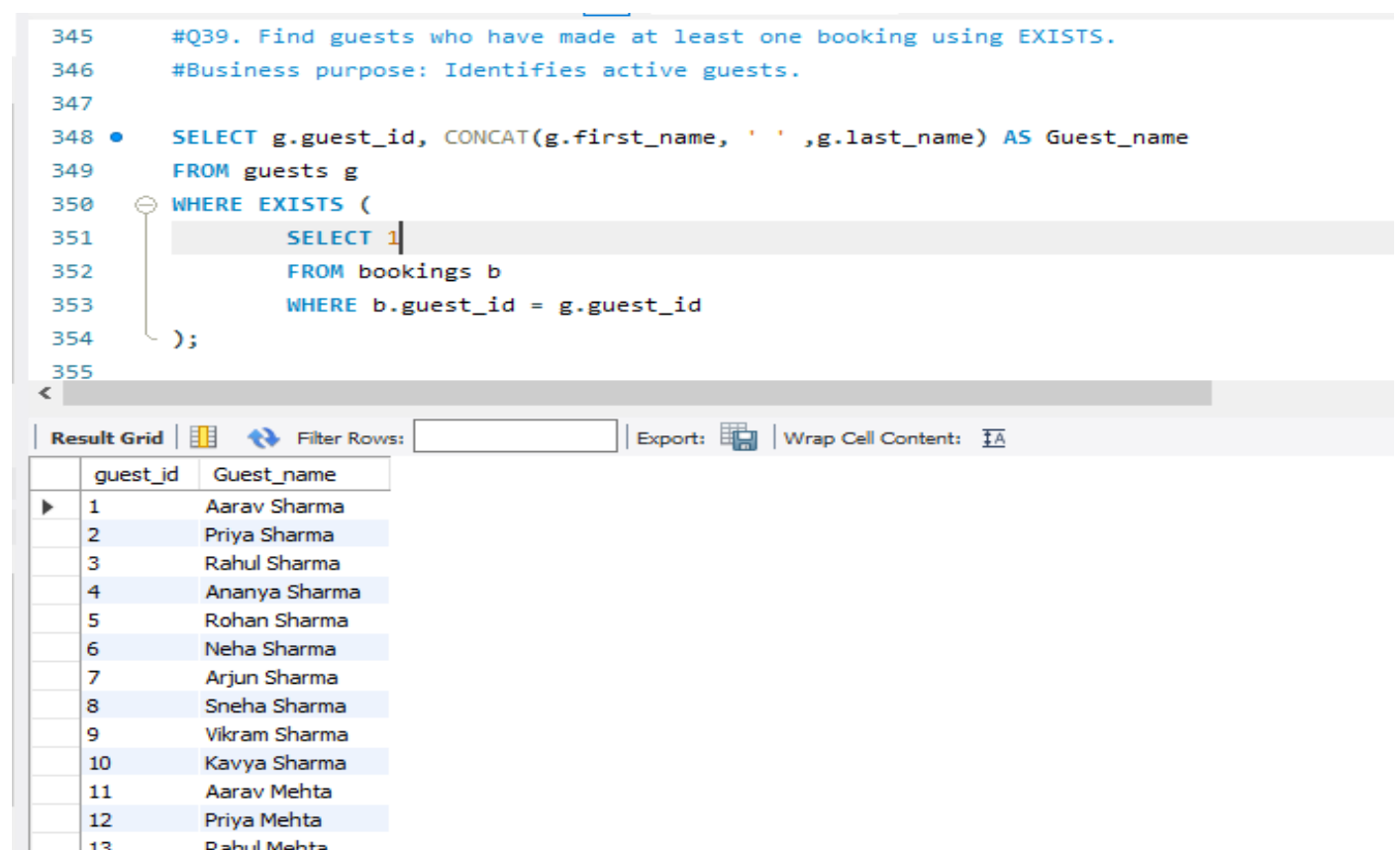
WHERE EXISTS (

    SELECT 1

    FROM bookings b

    WHERE b.guest_id = g.guest_id );
```

OUTPUT(SCREENSHOT)



```
345 #Q39. Find guests who have made at least one booking using EXISTS.
346 #Business purpose: Identifies active guests.
347
348 • SELECT g.guest_id, CONCAT(g.first_name, ' ', g.last_name) AS Guest_name
349 FROM guests g
350 WHERE EXISTS (
351     SELECT 1
352     FROM bookings b
353     WHERE b.guest_id = g.guest_id
354 );
355
```

	guest_id	Guest_name
▶	1	Aarav Sharma
	2	Priya Sharma
	3	Rahul Sharma
	4	Ananya Sharma
	5	Rohan Sharma
	6	Neha Sharma
	7	Arjun Sharma
	8	Sneha Sharma
	9	Vikram Sharma
	10	Kavya Sharma
	11	Aarav Mehta
	12	Priya Mehta
	13	Rahul Mehta

Q40. Find guests who have never made a booking using NOT EXISTS.

#Business purpose: Useful for re-engagement campaigns.

```
SELECT g.guest_id, CONCAT(g.first_name, ' ', g.last_name) AS
Guest_name

FROM guests g

WHERE NOT EXISTS (

    SELECT 1

    FROM bookings b

    WHERE b.guest_id = g.guest_id

);
```

OUTPUT(SCREENSHOT)

The screenshot displays a SQL IDE interface. The top section shows the SQL query being executed, with line numbers 357 through 367. The query is:
#Q40. Find guests who have never made a booking using NOT EXISTS.
#Business purpose: Useful for re-engagement campaigns.
SELECT g.guest_id, CONCAT(g.first_name, ' ', g.last_name) AS Guest_name
FROM guests g
WHERE NOT EXISTS (
 SELECT 1
 FROM bookings b
 WHERE b.guest_id = g.guest_id
);
The bottom section shows the 'Result Grid' with two columns: 'guest_id' and 'Guest_name'. The grid is currently empty, indicating that the query has not yet returned any results or that the results are not visible in the current view.

Q41. Show bookings whose total amount is greater than the average booking amount.

#Business purpose: Identifies unusually high-value reservations.

```
SELECT *  
  
FROM bookings  
  
WHERE total_amount > (  
  
        SELECT AVG(total_amount)  
  
        FROM bookings  
  
);
```

OUTPUT(SCREENSHOT)

369 #Q41. Show bookings whose total amount is greater than the average booking amount.
370 #Business purpose: Identifies unusually high-value reservations.
371
372 • SELECT *
373 FROM bookings
374 WHERE total_amount > (
375 SELECT AVG(total_amount)
376 FROM bookings
377);
378
379

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	booking_id	guest_id	room_id	check_in_date	check_out_date	number_of_guests	booking_status	total_amount	booking_source
	4	28	4	2025-01-09	2025-01-14	1	Confirmed	44800.00	Website
	5	35	15	2025-01-11	2025-01-17	2	Checked-In	33000.00	Website
▶	8	56	8	2025-01-17	2025-01-20	1	Confirmed	26160.00	Website
	9	63	19	2025-01-19	2025-01-23	2	Checked-In	24640.00	Website
	10	70	30	2025-01-21	2025-01-26	3	Cancelled	20000.00	Website
	16	112	16	2025-02-02	2025-02-07	1	Confirmed	41200.00	Website
	17	119	27	2025-02-04	2025-02-10	2	Checked-In	34980.00	Website
	20	140	20	2025-02-10	2025-02-13	1	Confirmed	24000.00	Website
	21	147	31	2025-02-12	2025-02-16	2	Checked-In	22660.00	Website
	22	4	2	2025-02-14	2025-02-19	3	Cancelled	21200.00	Website
	28	46	28	2025-02-26	2025-03-03	1	Confirmed	43600.00	Website
	29	53	39	2025-02-28	2025-03-06	2	Checked-In	36960.00	Website
	32	74	32	2025-03-06	2025-03-09	1	Confirmed	25440.00	Website

Q42. Find the highest-value booking for each guest.

#Business purpose: Identifies each guest's largest transaction.

```
SELECT *  
FROM bookings b  
WHERE total_amount = (  
    SELECT MAX(b2.total_amount)  
    FROM bookings b2  
    WHERE b2.guest_id = b.guest_id  
)  
ORDER BY total_amount desc;
```

OUTPUT(SCREENSHOT)

```

382 #Q42. Find the highest-value booking for each guest.
383 #Business purpose: Identifies each guest's largest transaction.
384
385 • SELECT *
386 FROM bookings b
387 WHERE total_amount = (
388     SELECT MAX(b2.total_amount)
389     FROM bookings b2
390     WHERE b2.guest_id = b.guest_id
391 )
392 ORDER BY total_amount desc;
393

```

Result Grid   Filter Rows: Edit:    Export/Import:   Wrap Cell Content: 

	booking_id	guest_id	room_id	check_in_date	check_out_date	number_of_guests	booking_status	total_amount	booking_source
	64	148	24	2025-05-09	2025-05-14	1	Confirmed	44800.00	Website
	124	118	4	2025-09-06	2025-09-11	1	Confirmed	44800.00	Website
	184	88	24	2026-01-04	2026-01-09	1	Confirmed	44800.00	Website
	244	58	4	2025-03-10	2025-03-15	1	Confirmed	44800.00	Website
	304	28	24	2025-07-08	2025-07-13	1	Confirmed	44800.00	Website
▶	364	148	4	2025-11-05	2025-11-10	1	Confirmed	44800.00	Website
	424	118	24	2025-01-09	2025-01-14	1	Confirmed	44800.00	Website
	484	88	4	2025-05-09	2025-05-14	1	Confirmed	44800.00	Website
	28	46	28	2025-02-26	2025-03-03	1	Confirmed	43600.00	Website
	88	16	8	2025-06-26	2025-07-01	1	Confirmed	43600.00	Website
	148	136	28	2025-10-24	2025-10-29	1	Confirmed	43600.00	Website

bookings62 x

Q43. Show rooms that have never appeared in the bookings table.

#Business purpose: Helps identify rooms with no booking activity.

```
SELECT *
FROM rooms r
WHERE NOT EXISTS (
    SELECT 1
    FROM bookings b
    WHERE b.room_id = r.room_id
);
```

OUTPUT(SCREENSHOT)

```
395 #Q43. Show rooms that have never appeared in the bookings table.
396 #Business purpose: Helps identify rooms with no booking activity.
397
398 • SELECT *
399 FROM rooms r
400 WHERE NOT EXISTS (
401     SELECT 1
402     FROM bookings b
403     WHERE b.room_id = r.room_id
404 );
405
```

[illegible]

LEVEL 6: SET OPERATIONS

Q44. List all guest IDs who either made a booking or wrote a review using UNION.

#Business purpose: Creates a combined list of engaged guests.

```
SELECT guest_id
FROM bookings

UNION

SELECT guest_id
FROM reviews;
```

OUTPUT(SCREENSHOT)

```
411 #Q44. List all guest IDs who either made a booking or wrote a review using UNION.
412 #Business purpose: Creates a combined list of engaged guests.
413
414 • SELECT guest_id
415 FROM bookings
416
417 UNION
418
419 SELECT guest_id
420 FROM reviews;
421
```

<

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	guest_id
▶	1
	2
	3
	4
	5
	6
	7
	8
	9
	10
	11

#Q45. List guest IDs who made a booking as well as wrote a review. Use INTERSECT if supported; otherwise solve with EXISTS or INNER JOIN.

#Business purpose: Identifies guests who interacted with the hotel in both ways

```
SELECT DISTINCT b.guest_id
FROM bookings b
INNER JOIN reviews r
ON b.guest_id = r.guest_id;
```

OUTPUT(SCREENSHOT)

```
423 #Q45. List guest IDs who made a booking as well as wrote a review. Use INTERSECT if supported; otherwise solve
424 # with EXISTS or INNER JOIN.
425 #Business purpose: Identifies guests who interacted with the hotel in both ways
426
427 • SELECT DISTINCT b.guest_id
428 FROM bookings b
429 INNER JOIN reviews r
430 ON b.guest_id = r.guest_id;
431
432
433
```

<

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

guest_id
3
6
9
12
15
18
21
24
27
30
33

Results 15 of 15

